Project 1 Report

3-Stage RAG System Specialised in Research Papers

Tran Hong Ha – 202416687
Hanoi University of Science and Technology

June 30, 2026

Contents

1 Introduction

Large language models (LLMs) are powerful tools for summarisation, question answering, and text generation, but they still have important limitations when they are used directly for research-paper understanding. A normal LLM may not know the newest papers, which may hallucinate by giving answers without grounding them in the exact evidence from the source document. This is a serious problem in research contexts, because users need answers that are accurate, traceable, and based on the content of specific papers rather than only on the model's internal knowledge.

Another problem appears when the input passage is too long. If a whole research paper or many long sections are sent directly to the LLM, the model has to read everything at once. This increases computation cost, may exceed the context window, and can make the model focus on irrelevant parts while missing the exact evidence needed for the question. Long inputs also make answers harder to control because the model must reason over too much unfiltered information.

To address this problem, this project proposes a Retrieval-Augmented Generation (RAG) system specialised for research papers [1]. Instead of asking the LLM to answer from memory alone, the system first retrieves relevant passages from a paper database, improves the quality of the retrieved evidence with reranking, and then feeds the most useful context into the LLM. The goal is to make the final answer more reliable, more relevant to the user's research question, and better supported by the original papers.

2 System Structure Overview

The proposed system is organised as the architecture shown in Figure 1. The user interacts with the system through a Gradio interface, which supports both uploading research papers and asking questions in a chat interface. Uploaded papers are processed by the chunking module, embedded, and stored in the ChromaDB vector database [4]. User questions are first sent to the semantic router, which decides whether the input is chit-chat or research-related.

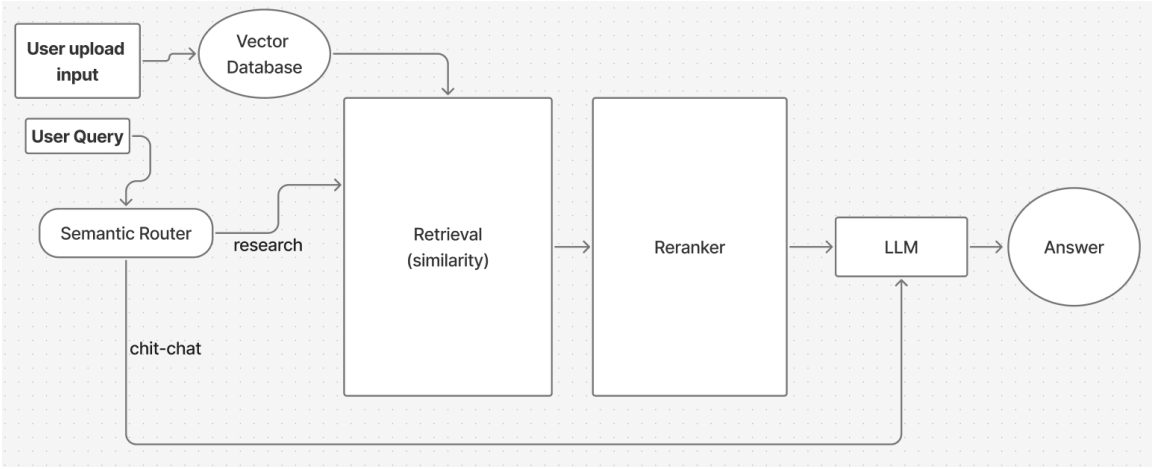

Figure 1: Overall architecture of the 3-stage RAG system for research-paper question answering.

For research-related questions, the system follows the main RAG pipeline:

User Interaction → Semantic Router → Retrieval → Reranker → LLM Generation → Answer

There are six main parts in the pipeline. First, **user interaction** includes both uploading files and submitting questions through the Gradio interface. Second, the **semantic router** decides whether the message is chit-chat or a research-related query. Third, the **retrieval module** searches ChromaDB for candidate chunks using embeddings and cosine similarity. Fourth, the **reranker** reorders the retrieved chunks so that the most relevant evidence is placed at the top. Fifth, the selected top chunks are **fed into Gemini** together with the user query. Finally, the system returns a grounded **answer** to the user through the chat interface.

The architecture therefore has two connected flows. The first flow is the document-ingestion flow, where uploaded papers are chunked, embedded, and indexed. The second flow is the question-answering flow, where the query is routed, relevant chunks are retrieved and reranked, and Gemini generates the final response from the selected evidence. The project source code is maintained in the author's GitHub repository [10].

3 Chunking Strategy

The chunking strategy is designed specifically for research papers, which usually have a clear hierarchy such as title, abstract, introduction, method, experiments, results, discussion, and conclusion. Before chunking, uploaded PDF files are first read and converted into Markdown using `pymupdf4llm` [9]. This Markdown representation helps preserve paper structure such as headings, paragraphs, and lists. The system then uses a recursive text splitter that follows this hierarchy as much as possible.

The retrieval process uses a hierarchical chunking idea. Large chunks represent main paper sections, while smaller child chunks represent detailed text parts inside those sections. This allows the system to preserve high-level structure while still retrieving precise evidence. The system can first identify relevant larger sections and then search or use smaller chunks inside them.

For embedding compatibility, the smaller chunks are limited to about 1000 characters with an overlap of 200 characters. The overlap helps avoid losing important information at chunk boundaries. This is useful because research-paper arguments often continue across sentences or paragraphs. The 1000-character length also helps chunks fit better into embedding models while keeping each chunk focused enough for retrieval and reranking.

The recursive chunking logic can be described with the following pseudocode:

```
Algorithm: RecursiveCharacterTextSplitter
Input: text, chunk_size = 1000, chunk_overlap = 200
```

```
Separators: [paragraph break, line break, sentence break, space, character]
Output: list of chunks

function split_text(text):
    return split(text, separators)

function split(text, separators):
    choose the first separator that appears in text
    raw_pieces = split text by the chosen separator
    good_pieces = empty list
    final_chunks = empty list

    for each piece in raw_pieces:
        if piece is empty:
            continue
        if length(piece) <= chunk_size:
            add piece to good_pieces
        else:
            if good_pieces is not empty:
                merge good_pieces into chunks with overlap
                add merged chunks to final_chunks
                clear good_pieces
            recursively split the oversized piece
            add recursive result to final_chunks

    if good_pieces is not empty:
        merge good_pieces into chunks with overlap
        add merged chunks to final_chunks

    return all non-empty final_chunks

function merge(pieces, separator):
    current_chunk = empty list
    chunks = empty list

    for each piece in pieces:
        if adding piece would exceed chunk_size:
            save current_chunk as one chunk
            remove old pieces from the front until overlap <= chunk_overlap
        add piece to current_chunk

    save the last current_chunk
    return all non-empty chunks
```

4 User Interaction

User interaction has two main forms: file upload and user query. In the file-upload flow, the user adds research papers or related documents through the Gradio upload interface. These files are then processed by the chunking strategy, embedded, and stored in ChromaDB so that they can be retrieved later.

In the query flow, the user enters a message through the chat interface. The message may be a simple conversational input, such as a greeting, or a research-related question, such as asking about a method, dataset, result, limitation, or comparison from a paper. Because not every input requires document retrieval, the system should not send all messages directly to the retrieval module.

For research-paper questions, the query is treated as an information need. The system must identify the meaning of the question and find evidence from the paper collection that can support the answer. For chit-chat messages, the system can respond directly without expensive retrieval.

5 Semantic Router

The first stage of the RAG system is the semantic router. It classifies the user query into two main types: **chit-chat** and **research-related**. This stage is important because it controls whether the RAG pipeline should be activated.

If the query is chit-chat, the system can produce a short conversational response without searching the research-paper database. If the query is research-related, the system forwards it to the retrieval stage. This design reduces unnecessary computation and helps the system focus retrieval only on questions that require evidence from documents.

In this project, the router is implemented as a lightweight semantic similarity router. Each route contains a route name and several sample utterances. During initialisation, the system encodes all sample utterances using `FastEmbedEncoder` with `sentence-transformers/all-MiniLM-L6-v2` [3]. These route embeddings are stored so that routing can be done quickly at query time.

When a new user query arrives, the router encodes the query into an embedding vector, normalises it, and compares it with the normalised embeddings of each route. The similarity score for a route is the average dot product between the query embedding and all sample embeddings for that route:

$$\text{score}(r, q) = \frac{1}{|S_r|} \sum_{s \in S_r} \frac{e_s}{\|e_s\|} \cdot \frac{e_q}{\|e_q\|}, \tag{1}$$

where S_r is the set of sample utterances for route r, e_s is the embedding of a route sample, and e_q is the query embedding. Because the vectors are normalised, the dot product is equivalent to cosine similarity. The router sorts all route scores and selects the route with the highest score. Its purpose is not to answer the question, but to decide whether the query should go to chit-chat

handling or to the research-paper RAG pipeline.

6 Retrieval Module

The second stage of the RAG system is the retrieval module. Its purpose is to find candidate passages that may contain the answer to the user's research question. The retrieval process uses three core components: **embeddings**, **cosine similarity**, and a **ChromaDB vector database**.

Each document chunk is converted into an embedding vector by an embedding model. The user query is also encoded into the same vector space. The system then computes cosine similarity between the query embedding and the stored chunk embeddings. Chunks with higher cosine similarity are considered more semantically related to the query and are returned as candidate evidence.

ChromaDB is used as the vector database for storing and searching embeddings. It allows the system to index paper chunks and retrieve the top matching chunks efficiently. The retrieval module is evaluated using the QASPER dataset [2], which is suitable because it contains question-answering examples based on academic papers.

The main retrieval metric is hit rate@k. It checks whether at least one relevant evidence chunk appears inside the top-k retrieved chunks. For a set of queries Q, it is calculated as:

$$\text{HitRate@k} = \frac{1}{|Q|} \sum_{q \in Q} \mathbf{1} \left[\text{Rel}(q) \cap \text{TopK}(q) \neq \emptyset \right], \tag{2}$$

where $\text{Rel}(q)$ is the set of relevant chunks for query q, $\text{TopK}(q)$ is the top-k retrieved list, and $\mathbf{1}[\cdot]$ returns 1 if the condition is true and 0 otherwise.

The embedding implementation follows a shared `BaseEmbedding` interface. This base class stores the model name and defines an `encode` method that every embedding model must implement. Using this common interface makes it possible to compare different embedding backends in the same retrieval pipeline without changing the rest of the system.

The first implementation is `FastEmbedding`, which uses `fastembed.TextEmbedding` [5]. It is designed for efficient benchmark experiments because it supports configurable model names, optional cache directories, optional local ONNX model paths, and automatic GPU usage when available. If GPU embedding fails, the class retries on CPU with a smaller batch size, making the retrieval experiment more robust.

The second implementation wraps `SentenceTransformer` [3]. It receives an `EmbeddingConfig`, validates that the model name is non-empty, loads the selected sentence-transformer model, and encodes either a single string or a list of strings. Together, these embedding classes allow the project to evaluate FastEmbed-compatible models and SentenceTransformer models under one common interface.

Figure 2 shows the uploaded embedding benchmark. The image compares the retrieval performance of several candidate embedding models on QASPER. The benchmark shows that `BAAI/bge` provides the strongest overall retrieval result among the tested options, so this project chooses `BAAI/bge` as the main embedding model for the retrieval module. However, the benchmark also shows that the hit rate@50 is only about 65%. This means that even when the system retrieves 50 candidate chunks, the correct evidence is still missing in many cases. Therefore, the need for a reranker becomes apparent: retrieval alone is not strong enough to guarantee that the best evidence is ranked near the top.

The project also experimented with fine-tuning an embedding model based on `bge-m3` using research query–document pairs. The goal was to adapt the embedding space more closely to scientific question answering, where queries and relevant paper passages can use specialised terminology. However, the metric results were not significantly different from the original model. The fine-tuned model only improved hit rate@100, while hit rate@50 became worse. This suggests that fine-tuning helped place some relevant chunks somewhere in a larger candidate pool, but it did not reliably move them into the more useful top-50 range. Therefore, the base BGE embedding model remains the safer choice for the current retrieval stage, and reranking is still needed to improve the ordering of retrieved candidates.

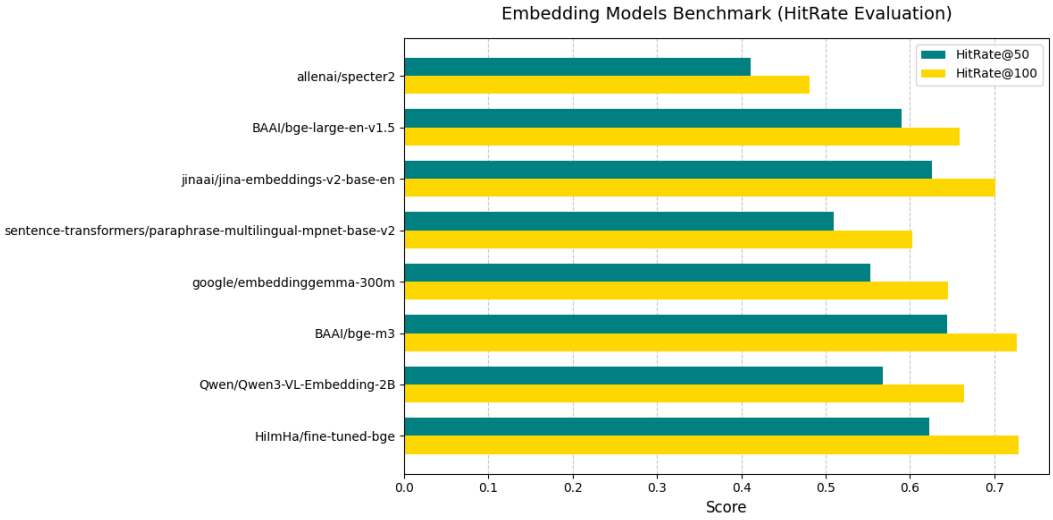

Figure 2: Embedding model benchmark for retrieval performance on QASPER.

7 Reranker Module

The third stage of the RAG system is the reranker module. After the retrieval module returns a list of candidate chunks, the reranker evaluates these candidates more carefully and reorders them according to their relevance to the query.

This stage is necessary because vector retrieval is fast but not always precise. A chunk may have high cosine similarity to the query while still not containing the exact answer. The reranker compares the query and each candidate chunk more directly, then promotes the chunks that are

most likely to contain useful evidence.

The reranker is also evaluated using QASPER. Figure 3 compares three reranker models with NDCG@25 and NDCG@35. NDCG is useful here because it measures not only whether relevant chunks are retrieved, but also whether they are placed near the top of the ranked list. Higher NDCG means the most useful evidence is closer to the positions that will later be sent to the LLM.

For a ranked list of k chunks, Discounted Cumulative Gain is:

$$\text{DCG@k} = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)}, \tag{3}$$

where rel_i is the relevance score of the chunk at rank i. NDCG normalises this value by the ideal ranking:

$$\text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}. \tag{4}$$

This means a reranker receives a higher score when it places highly relevant evidence earlier in the list.

The benchmark shows that `BAAI/bge-reranker-v2-m3` performs best overall. It obtains the highest NDCG@25 and NDCG@35 among the tested models, slightly ahead of the Jina multilingual reranker and clearly ahead of the MiniLM cross-encoder. Therefore, this project chooses the BGE reranker as the main reranker. This choice is also consistent with the embedding-stage decision to use a BAAI/BGE model family, which keeps the retrieval and reranking components semantically aligned.

For the reranker pool size, the system should retrieve a wider candidate pool from ChromaDB and then rerank only the most promising candidates. Since the retrieval hit rate@50 is about 65%, using a retrieval pool of 50 keeps enough candidates for recall. The reranker pool is then set to **k = 35**. This value matches the NDCG@35 benchmark, preserves more evidence than k = 25, and is still much cheaper than reranking every retrieved chunk.

After reranking, the system should pass only the **top 5 chunks** to the LLM. The NDCG result shows that the BGE reranker is effective at moving relevant evidence toward the top of the reranked list, so the highest-ranked chunks should contain the most useful context. Using five chunks gives the LLM enough evidence to answer research questions while avoiding excessive context length, duplicated information, and irrelevant noise from lower-ranked chunks.

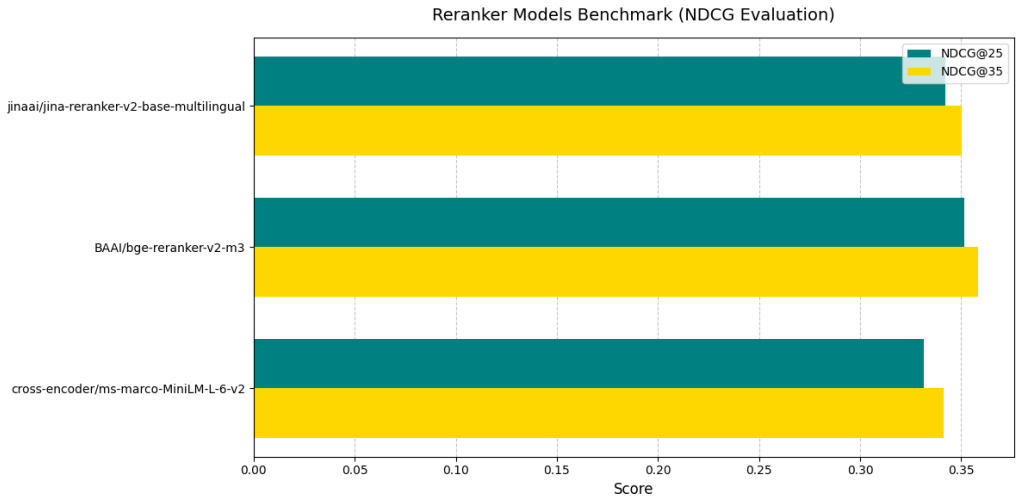

Figure 3: Reranker benchmark using NDCG@25 and NDCG@35 on QASPER.

8 LLM Context Feeding and Answer Generation

After reranking, the top-ranked chunks are selected as context for the LLM. The system combines the user query with the retrieved evidence and sends both to the generation model. This helps the LLM answer based on the paper content rather than relying only on its pre-trained knowledge.

In this project, the generation model is implemented with Google's Gemini API [6]. The LLM wrapper creates a `genai.Client` using an API key and uses `gemini-2.5-flash` as the default model. A system instruction, imported from `SYS_INSTRUCTION`, is attached to every interaction so that the model follows the desired research-assistant behaviour. The wrapper exposes a `create_content` function that receives the final prompt and sends it to Gemini with a configurable `thinking_level`, which is set to `low` by default for faster responses.

The final prompt contains the user query and the top 5 reranked chunks. The prompt should encourage Gemini to use the provided context, avoid unsupported claims, and clearly answer the user's question. If the retrieved context is insufficient, the model should indicate uncertainty instead of inventing information. This behaviour is especially important for research-paper question answering, where factual accuracy and evidence grounding are required.

The final answer is then returned to the user through the interface. Ideally, the answer should be concise, relevant, and connected to the retrieved passages from the papers.

9 Gradio Simulation Interface

Gradio is used as the simulation library for the project [7]. It provides a simple interface where users can enter queries and observe the system response. This is useful for testing the full RAG

pipeline because the developer can interact with the system in a realistic way without building a complete production application.

Through the Gradio interface, the project can demonstrate query routing, retrieval results, reranking behaviour, and final LLM answers. It also makes debugging easier because intermediate outputs can be displayed during experimentation.

Figure 4 shows the file-upload interface. This screen is used to add research papers or related files into the system. After uploading, the documents can be processed by the chunking module, embedded, stored in ChromaDB, and later retrieved during question answering.

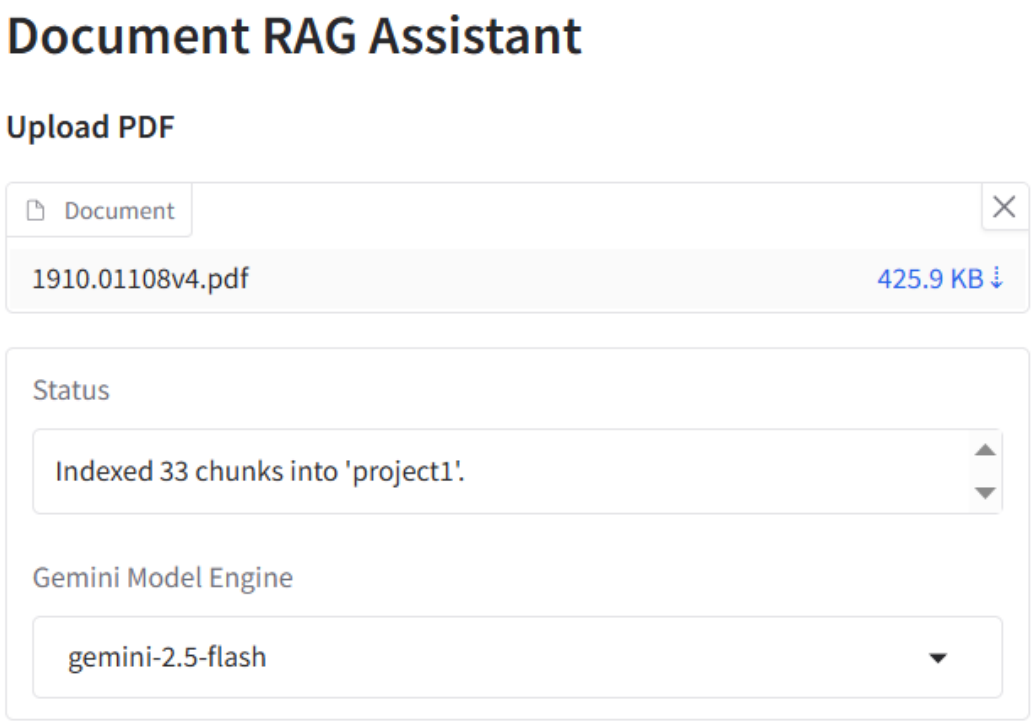

Figure 4: File-upload interface for adding research documents to the local RAG system.

Figure 5 shows the chat interface. This screen is where the user enters questions and receives answers generated by the RAG pipeline. It represents the main interaction flow: the user asks a question, the semantic router decides whether retrieval is needed, and the final answer is displayed in the chat window.

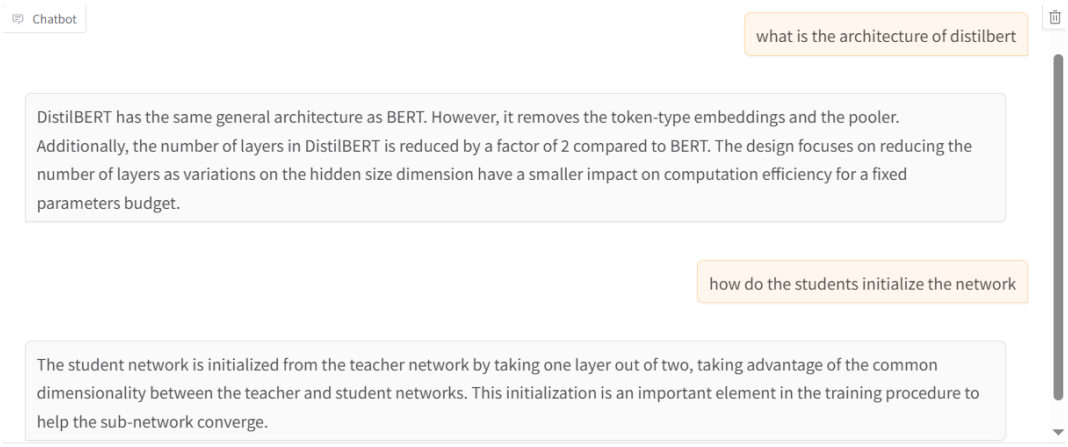

Figure 5: Chat interface for asking questions and receiving RAG-based answers.

10 Limitations and Future Work

At the current stage, the main limitation is that retrieval performance is still imperfect, as shown by the hit rate@50 result. Even when the system retrieves many candidate chunks, some relevant evidence may still be missing or ranked too low. Another limitation is that the final answer quality depends on the selected embedding model, reranker, chunking strategy, and LLM prompt design.

The current system also has several application-level limitations. First, it does not include a reflection or memory module, so the LLM cannot see previous conversations and each user query is treated mostly as an independent turn. Second, the application currently runs only locally, which makes it suitable for demonstration and testing but not yet ready for multi-user deployment. Third, although the interface is local, the answer generation still uses an external LLM API. This creates a possible security and privacy concern because prompts and retrieved paper context may be sent outside the local machine.

Future work will focus on improving retrieval accuracy, testing stronger reranker models, refining the hierarchical chunking strategy, and adding better evaluation metrics for both evidence retrieval and final answer quality. The Gradio interface can also be extended to show intermediate retrieval and reranking results more clearly for debugging and demonstration.

I should also implement a reflection module for previous conversations. This would allow the system to remember earlier user questions, selected papers, and research preferences across turns. Another important direction is to replace the external API model with a local LLM to improve privacy and reduce dependence on cloud services. Possible local model choices include Llama 3, Mistral, Qwen, and Gemma models, depending on the available hardware and required response quality.

11 Conclusion

This project designs a 3-stage RAG system specialised in research papers, supported by related backend RAG implementation references [8]. The three core stages are semantic routing, retrieval, and reranking before final LLM answer generation. The system addresses the limitations of normal LLMs by grounding answers in retrieved paper evidence. ChromaDB, embeddings, cosine similarity, reranking, hierarchical chunking, and Gradio simulation are combined to create a more reliable research-paper question-answering workflow.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 2020.

[2] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. A Dataset of Information-Seeking Questions and Answers Anchored in Research Papers. *Proceedings of NAACL*, 2021.

[3] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP-IJCNLP*, 2019.

[4] Chroma. ChromaDB Documentation. Vector database documentation for embedding storage and similarity search.

[5] Qdrant. FastEmbed Documentation. Lightweight text embedding library for retrieval applications.

[6] Google. Gemini API Documentation. Developer documentation for Gemini model access through the Google GenAI SDK.

[7] Gradio Team. Gradio Documentation. Python library for building machine-learning web interfaces.

[8] Bangoc123. Retrieval Backend with RAG. GitHub repository: `https://github.com/bangoc123/retrieval-backend-with-rag.git`.

[9] PyMuPDF. PyMuPDF4LLM. GitHub repository: `https://github.com/pymupdf/pymupdf4llm.git`.

[10] Tran Hong Ha. Rag Chatbot Project Repository. GitHub repository: `https://github.com/CRedRiver/Rag_Chatbot.git`.